

Diretrizes de Versionamento: Boas Práticas, Commits e Resolução de Conflitos

1. Padrão de Mensagens de Commit (Conventional Commits)

Para garantir um histórico de alterações legível e automatizável, todas as mensagens de commit devem seguir a especificação do Conventional Commits. O formato padrão é estruturado da seguinte forma:

Plaintext

() : <descrição breve em letras minúsculas>

Tipos Permitidos no Projeto

`feat`: Introdução de uma nova funcionalidade ou componente (ex:

`feat(header): adiciona menu mobile expansível`).

`fix`: Correção de um bug ou comportamento incorreto (ex: `fix(carousel):`

corrige quebra de layout em resoluções menores que 360px).

`style`: Alterações que não afetam o comportamento do código (espaçamentos, formatação, correções de CSS/Design Tokens).

`refactor`: Alteração no código que não corrige bug nem adiciona funcionalidade, mas melhora a estrutura (ex: `refactor(form): simplifica gerenciamento de estado do useState`).

`docs`: Modificações exclusivas na documentação (ex: `docs(readme): atualiza instruções de build local`).

2. Boas Práticas de Versionamento

Commits Atômicos

Cada commit deve representar uma única unidade de trabalho lógica e funcional. Evite acumular alterações em diferentes seções da página (como mexer no FAQ e no formulário ao mesmo tempo) para gerar um único commit massivo. Commits menores facilitam a revisão de código e a reversão de problemas isolados.

Fluxo de Trabalho (Git Flow Simplificado)

A branch `main` representa o código em produção e nunca deve receber commits diretos dos desenvolvedores.

Todo desenvolvimento deve ocorrer em branches de funcionalidade (feature branches), criadas a partir da `main` atualizada.

Padrão de nomenclatura para branches: `feat/nome-da-secao` ou `fix/nome-do-bug`.

3. Como Evitar Conflitos de Código (Merge Conflicts)

Em uma equipe de 3 desenvolvedores atuando na mesma Landing Page, a sobreposição de arquivos é o principal fator gerador de conflitos. Adote as seguintes posturas preventivas:

Modularização Estrita: O projeto deve ser dividido em arquivos de componentes isolados dentro de uma pasta `/components`. Os desenvolvedores não devem editar o arquivo principal `App.jsx` ou `index.css` simultaneamente. Cada um trabalha em seu próprio arquivo (ex: `Hero.jsx`, `FAQ.jsx`).

Atualização Diária: Antes de iniciar o desenvolvimento no dia, ou antes de abrir um Pull Request, puxe as alterações mais recentes da branch principal para a sua máquina local:

Bash

```
git checkout main
git pull origin main
git checkout sua-branch
git merge main
```

3. Comunicação de Alterações Globais: Se houver necessidade de alterar arquivos compartilhados (como definições de cores globais ou o arquivo de rotas/Vite), avise a equipe antes de realizar a modificação.

4. Como Resolver Conflitos de Forma Segura

Caso o Git identifique alterações conflitantes no mesmo arquivo durante um processo de merge, o fluxo de resolução deve seguir rigorosamente estes passos:

Passo 1: Identificar os arquivos afetados

O terminal indicará quais arquivos falharam no merge automático. O comando `git status` listará esses arquivos sob a seção *Unmerged paths*.

Passo 2: Analisar os marcadores de conflito

Abra o arquivo afetado no editor de código (Neovim/VS Code). O Git insere marcadores visuais delimitando as diferenças:

```
<<<<<< HEAD
// Seu código local (as alterações que você fez na sua branch)
const color = "var(--primary)";
=====
// Código remoto (as alterações que já foram aprovadas na main)
const color = "var(--ink-soft)";
>>>>>> main
```

Passo 3: Escolher a versão correta e limpar o arquivo

Avalie semanticamente qual alteração deve prevalecer ou se ambas deve

Remova manualmente todas as linhas de marcação (<<<<<<, =====, >>>

O arquivo deve conter apenas código válido após a edição.

Passo 4: Concluir o processo de Merge

Após limpar e salvar o arquivo, avise o Git que o conflito foi resolvido
Bash

```
git add arquivo_resolvido.jsx
git commit -m "fix: resolve conflitos de merge com a branch main"
git push origin sua-branch
```